CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

4. Small Worlds

Lecture 12: Characteristic Path Length and Clustering Coefficient

Many real world networks are **small world networks**, where most pairs of nodes are only a few steps away from each other, and where nodes tend to form cliques, i.e., subgraphs having all nodes connected to each other.

For example, MathSciNet (https://mathscinet-ams-org.libgate.library.nuigalway.ie/mathscinet/freeTools.html?version=2) allows its users to explore distances between authors in the collaborations network. The distance of an author to Erdös is know as this author's Erdös number (https://en.wikipedia.org/wiki/Erd%C5%91s_number).

We introduce **three network attributes** that measure these small-world effects:

- the **characteristic path length** L, defined a the average length of all shortest paths in the network;
- the **transitivity** T, defined as the proportion of triads that form triangles;
- the **clustering coefficient** C, defined as the average node clustering coefficient.

In terms of these attributes, a network is called a **small world network** if it has

1. a small **average shortest path length** L (scaling with $\log n$, where n is the number of nodes), and
2. a high **clustering coefficient** C.

It turns out that ER random networks do have a small average shortest path length, but not a high clustering coefficient. This observation justifies the need for a different model of random networks, if they are to be used to model the clustering behavior of real world networks.

In [1]:
```
import networkx as nx
opts = { "with_labels" : True, "node_color": 'y' }
```

Characteristic Path Length

We have seen how BFS can determine the length of a shortest path from a given node x to any node y in a **connected network**. An application to all nodes x yields the shortest distances between all pairs of nodes.

Let $\mathcal{D} = (d_{ij})$ be the **distance matrix** of a connected graph $G = (X, E)$, whose entry d_{ij} is the length of the shortest path from node $i \in X$ to node $j \in X$. (Hence $d_{ii} = 0$ for all i.) There are a number of graph (and node) attributes that can be defined in terms of this matrix.

Definition. Let $G = (X, E)$ be a connected graph.

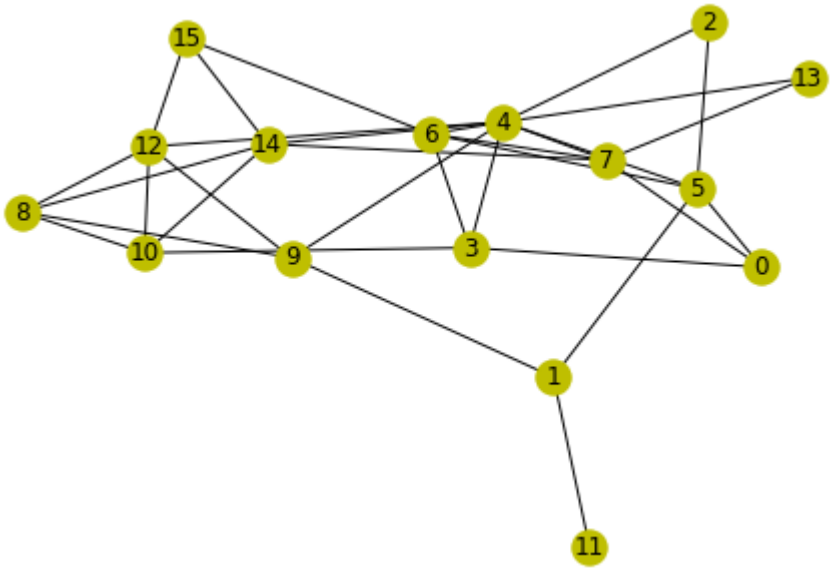
- The **eccentricity** e_i of a node $i \in X$ is the maximum distance between i and any other vertex in G,
$$e_i = \max_j d_{ij}.$$
- The **graph radius** R is the minimum eccentricity,
$$R = \min_i e_i.$$
- The **graph diameter** D is the maximum eccentricity,
$$D = \max_i e_i.$$
- The **characteristic path length** L of G is the average distance between pairs of distinct nodes,
$$L = \frac{1}{n(n-1)} \sum_{i \neq j} d_{ij}.$$

- **Fact:** The characteristic path length of a random graph $G(n, m)$, or $G(n, p)$ is
$$L = \frac{\ln n}{\ln \langle k \rangle}.$$

So if $n = 16$ and $m = 32$, then the average node degree in $G(n, m)$ is $\langle k \rangle = \frac{2m}{n} = 4$, and, approximately, $L = \frac{\log_2 16}{\log_2 4} = 2$.

Let's find a small connected graph. (Loop until it's connected ...)

In [2]:
```
n, m = 16, 32
while True:
    G = nx.gnm_random_graph(n, m)
    if nx.is_connected(G):
        break
nx.draw(G, **opts)
```



Compute **all** shortest path lengths with the `shortest_path_length` function provided by `networkx` .

In [3]:
```
dist = dict(nx.shortest_path_length(G))
dist = [[dist[i][j] for j in range(n)] for i in range(n)]
dist
```

Out[3]:
```
[[0, 2, 2, 1, 2, 1, 2, 1, 3, 3, 2, 3, 3, 2, 2, 3],
 [2, 0, 2, 3, 2, 1, 2, 3, 2, 1, 3, 1, 2, 3, 3, 3],
 [2, 2, 0, 2, 1, 1, 2, 2, 3, 2, 3, 3, 2, 2, 2, 3],
 [1, 3, 2, 0, 1, 2, 1, 2, 2, 2, 1, 4, 2, 2, 2, 2],
 [2, 2, 1, 1, 0, 1, 1, 1, 2, 1, 2, 3, 1, 1, 1, 2],
 [1, 1, 1, 2, 1, 0, 1, 2, 3, 2, 3, 2, 2, 2, 2, 2],
 [2, 2, 2, 1, 1, 1, 0, 1, 3, 2, 2, 3, 2, 2, 2, 1],
 [1, 3, 2, 2, 1, 2, 1, 0, 2, 2, 2, 4, 2, 1, 1, 2],
 [3, 2, 3, 2, 2, 3, 3, 2, 0, 1, 1, 3, 1, 3, 1, 2],
 [3, 1, 2, 2, 1, 2, 2, 2, 1, 0, 2, 2, 1, 2, 2, 2],
 [2, 3, 3, 1, 2, 3, 2, 2, 1, 2, 0, 4, 1, 3, 1, 2],
 [3, 1, 3, 4, 3, 2, 3, 4, 3, 2, 4, 0, 3, 4, 4, 4],
 [3, 2, 2, 2, 1, 2, 2, 2, 1, 1, 1, 3, 0, 2, 2, 1],
 [2, 3, 2, 2, 1, 2, 2, 1, 3, 2, 3, 4, 2, 0, 2, 3],
 [2, 3, 2, 2, 1, 2, 2, 1, 1, 2, 1, 4, 2, 2, 0, 1],
 [3, 3, 3, 2, 2, 2, 1, 2, 2, 2, 2, 4, 1, 3, 1, 0]]
```

In [4]:
```
eccentricity = [max(d) for d in dist]
eccentricity
```

Out[4]:
```
[3, 3, 3, 4, 3, 3, 3, 4, 3, 3, 4, 4, 3, 4, 4, 4]
```

`networkx` has a function `eccentricity` which computes a dictionary of eccentricities.

In [5]:
```
print(nx.eccentricity(G))
```

```
{0: 3, 1: 3, 2: 3, 3: 4, 4: 3, 5: 3, 6: 3, 7: 4, 8: 3, 9: 3, 10: 4, 1
1: 4, 12: 3, 13: 4, 14: 4, 15: 4}
```

The extreme values of the eccentricity are the radius and the diameter of the network.

In [6]:
```
radius = min(eccentricity)
diameter = max(eccentricity)
radius, diameter
```

Out[6]:
```
(3, 4)
```

The characteristic path length L is the sum of all entries in $\mathcal{D}$, divided by the number of pairs of distinct nodes.

In [7]:
```
cpl = sum([sum(d) for d in dist]) / n / (n - 1)
cpl
```

Out[7]:
```
2.0416666666666665
```

`networkx` computes L with a function `average_shortest_path_length` .

In [8]:
```
nx.average_shortest_path_length(G)
```

Out[8]: 2.0416666666666665

In [9]:
```
from math import log
kavg = sum(dict(G.degree()).values()) / n
log(n) / log(kavg)
```

Out[9]: 2.0

In [10]:
```
kavg
```

Out[10]: 4.0

Definition (Small-world behaviour). A network $G = (X, E)$ is said to exhibit a **small world behaviour** if its characteristic path length L grows proportionally to the logarithm of the number n of nodes of G:

$$L \sim \ln n.$$

In this sense, the ensembles $G(n, m)$ and $G(n, p)$ of random graphs do exhibit small world behavior (as $n \to \infty$).

Clustering

In contrast to random graphs, real world networks contain **many triangles**: it is not uncommon that a friend of one of my friends is my friend, too. This **degree of transitivity** can be measured in several different ways.

Definition (Graph transitivity). A **triad** is a tree of 3 nodes or, equivalently, a graph consisting of 2 adjacent edges (and the nodes they connect). The transitivity T of a graph $G = (X, E)$ is the proportion of **transitive** triads, i.e., triads which are subgraphs of **triangles**:

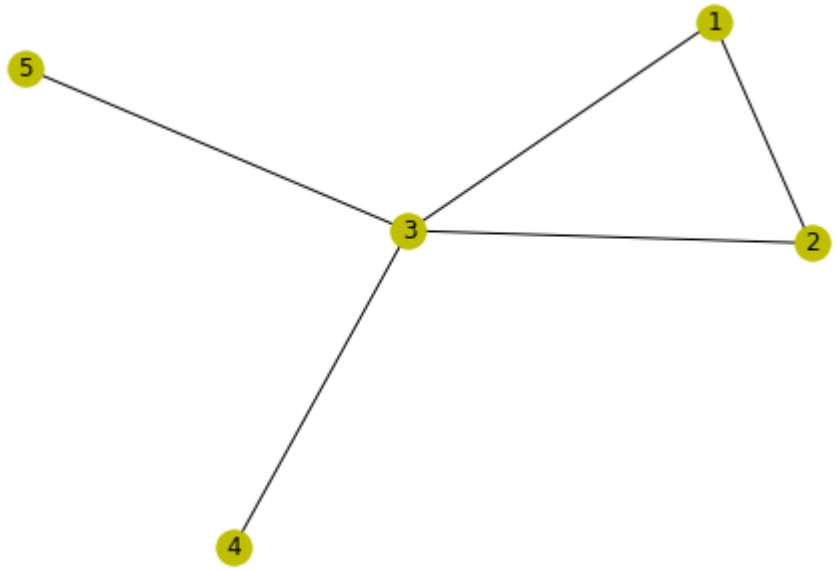
$$T = \frac{3n_\Delta}{n_\wedge},$$

where n_Δ is the number of triangles in G, and $n_\wedge$ is the number of triads.

By definition, $0 \leq T \leq 1$.

Example.

In [11]:
```
G = nx.Graph(((1,2), (2,3), (3,1), (3,4), (3,5)))
nx.draw(G, **opts)
```



The function `nx.triangles(G)` returns a `python` dictionary reporting for each node of the graph `G` the number of triangles it is contained in.

In [12]:
```
print(nx.triangles(G))
```

```
{1: 1, 2: 1, 3: 1, 4: 0, 5: 0}
```

Overall, each triangle in `G` is thus accounted for 3 times, once for each of its vertices. The following sum determines this number $3n_{\Delta}$.

In [13]:
```
triple_nr_triangles = sum(nx.triangles(G).values())
print(triple_nr_triangles)
```

```
3
```

The number $n_{\wedge}$ of triads in `G` can be determined from the graph's degree sequence, as each node of degree k is the central node of exactly $\binom{k}{2}$ triads. (Why?)

In [14]:
```
print(G.degree())
print({k : v * (v-1) // 2 for k, v in dict(G.degree()).items()})
nr_triads = sum([v * (v-1) // 2 for v in dict(G.degree()).values()])
nr_triads
```

```
[(1, 2), (2, 2), (3, 4), (4, 1), (5, 1)]
{1: 1, 2: 1, 3: 6, 4: 0, 5: 0}
```

Out[14]: 8

The transitivity T of `G` is the quotient of these two quantities, $T = 3n_\Delta/n_\wedge$, which `networkx` computes with a function `transitivity`.

In [15]:
```
print(triple_nr_triangles / nr_triads )
print(nx.transitivity(G))
```

```
0.375
0.375
```

- The transitivity of a $G(n, p)$ **random graph** is
$$T = p,$$
the probability of any edge as third edge in a triangle. (Or: Compute $3n_\Delta/n_\wedge$ using the explicit formulas from the previous lecture: $n_\Delta = \binom{n}{3}p^3$ and $n_\wedge = 3\binom{n}{3}p^2$.)

The concept of **clustering** measures the transitivity of a node, or of an entire graph in a different way.

Definition (Clustering coefficient). For a node $i \in X$ of a graph $G = (X, E)$, denote by G_i the subgraph induced on the neighbours of i in G, and by $m(G_i)$ its number of edges.

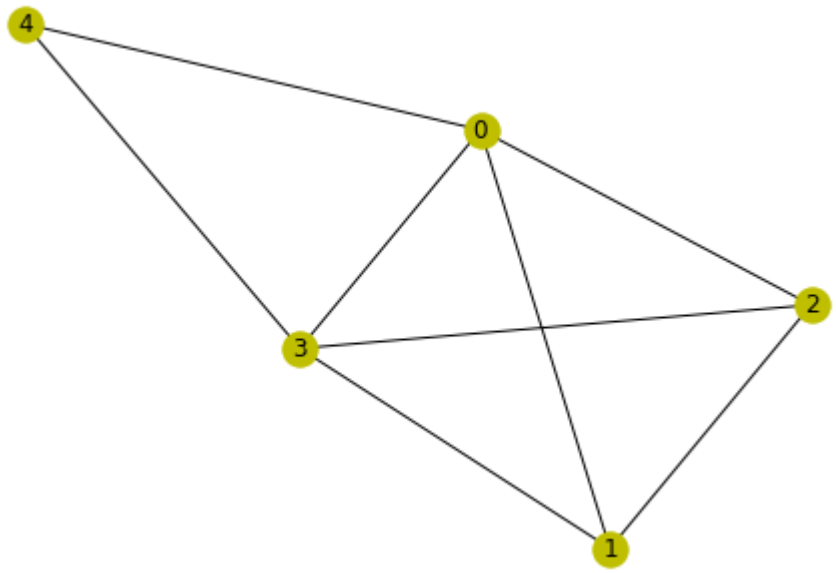
The **node clustering coefficient** c_i of node i is defined as
$$c_i = \begin{cases} \binom{k_i}{2}^{-1} m(G_i), & k_i \geq 2, \\ 0, & \text{else.} \end{cases}$$

The **graph clustering coefficient** C of G is the average node clustering coefficient,
$$C = \langle c \rangle = \frac{1}{n}\sum_{i=1}^{n} c_i.$$

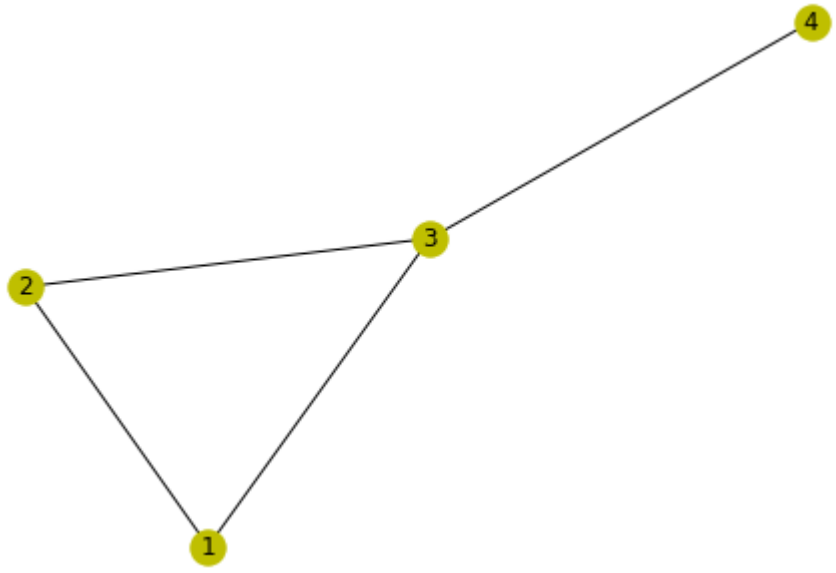
By definition, $0 \leq c_i \leq 1$ for all nodes $i \in X$, and $0 \leq C \leq 1$.

Example.

In [16]:
```
G = nx.Graph([(0,1), (0,2), (0,3), (0,4), (1,2), (1,3), (2,3), (3,4)])
nx.draw(G, **opts)
```



In [17]:
```
N = nx.neighbors(G, 0)
S = G.subgraph(list(N))
nx.draw(S, **opts)
```



In [18]:
```
nS = S.number_of_nodes()
nS_choose_2 = nS * (nS - 1) // 2
mS = S.number_of_edges()
print(nS, mS, mS / nS_choose_2 )
```

```
4 4 0.6666666666666666
```

In [19]:
```
nx.clustering(G)
```

Out[19]:
```
{0: 0.6666666666666666, 1: 1.0, 2: 1.0, 3: 0.6666666666666666, 4: 1.0}
```

In [20]:
```
nx.average_clustering(G)
```

Out[20]:
```
0.8666666666666666
```

- The **node clustering coefficient** of any node i in a $G(n,p)$ **random graph** is $c_i = p$. (In any selection of potential edges, by construction a proportion of p is present in the random graph; this is true in particular for the $\binom{k}{2}$ potential edges between the k neighbors of a node of degree k.)
- Thus the **graph clustering coefficient** of a $G(n,p)$ **random graph** is
$$C = p.$$
- Note that when $p(n) = \langle k \rangle n^{-1}$ for a fixed expected average degree $\langle k \rangle$ then $C = \langle k \rangle / n \to 0$ for $n \to \infty$: in large random graphs the number of triangles is negligible.
- In real world networks, one often observes that $C/\langle k \rangle$ does not depend. on n (as $n \to \infty$)

Clustering vs Transitivity

For a node $i \in X$, denote by $n_i^\wedge = \binom{k_i}{2}$ the number of triads containing i as their central node, and by n_i^Δ the actual number of triangles containing i.

Then the node clustering coefficient is $c_i = n_i^\Delta / n_i^\wedge$, or $n_i^\Delta = n_i^\wedge c_i$.

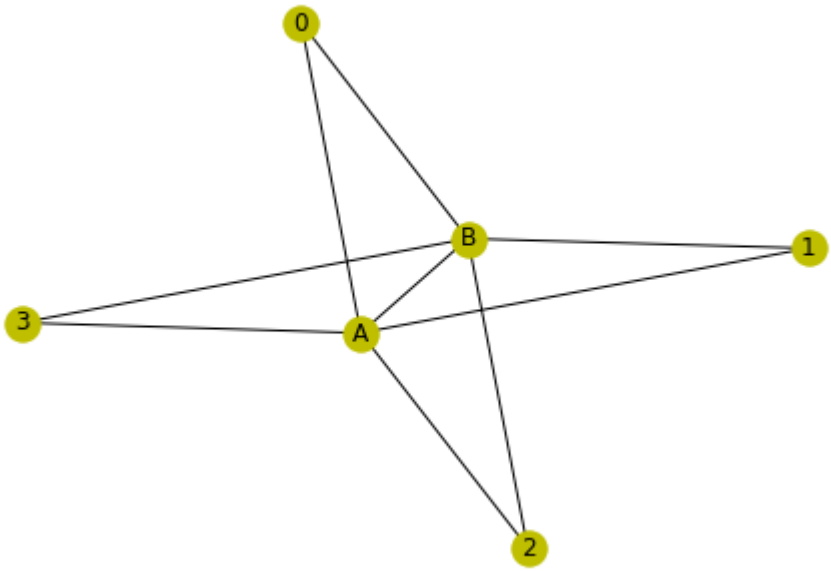
Moreover $3n_\Delta = \sum_i n_i^\Delta$ and $n_\wedge = \sum_i n_i^\wedge$.

It follows that
$$T = \frac{3n_\Delta}{n_\wedge} = \frac{1}{n_\wedge} \sum_i n_i^\wedge c_i$$
in contrast to
$$C = \frac{1}{n} \sum_i c_i,$$
C is the (plain) **average** of the node clustering coefficients, whereas T is a **weighted average** of node clustering coefficients, giving higher weight to high degree nodes.

The following example illustrates how C and T are different measures: as $n \to \infty$ here, $T \to 0$ while $C \to 1$.

In [21]:
```
n = 4
G = nx.Graph(["AB"])
G.add_edges_from([(x, k) for x in "AB" for k in range(n)])

nx.draw(G, **opts)
```



In [22]:
```
nx.average_clustering(G), nx.transitivity(G)
```

Out[22]:
```
(0.7999999999999999, 0.5)
```

Code Corner

networkx

- `shortest_path_length` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr
- `eccentricity` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr
- `triangles` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr
- `transitivity` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr
- `clustering` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr

- `average_clustering` : [doc] (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithr

Exercises

1. What are the characteristic path length L, the transitivity T, and the clustering coefficient C of the Peterson graph?
2. What are the characteristic path length L, the transitivity T, and the clustering coefficient C of the Florentine families marital graph?
3. What is the transitivity and what is the clustering coefficient of a complete graph on n nodes?
4. What is the transitivity and what is the clustering coefficient of a tree on n nodes?
5. Design an experiment with random graphs to verify the predicted characteristic path length.
6. Design an experiment with random graphs to verify the predicted graph clustering coefficient.

In [ ]: